



UNIVERSITÀ DI PISA

UNIVERSITÀ DI PISA
LAUREA MAGISTRALE INFORMATICA UMANISTICA

Report Progetto esame a.a. 25/26

Linguistica Computazionale II [513LL]

Studente

MICHELA DEODATI 597983

Professori

FELICE DELL'ORLETTA

GIULIA VENTURI

27/07/2026

CONTENTS

1	Dataset	1
1.1	Subset del dataset	1
1.2	Features	2
2	Sviluppo e Discussione	6
2.1	Classificatore basato su SVM lineari e n-grammi	6
2.2	Classificatore basato su Support Vector Machine (SVM) lineari e informazioni linguistiche non lessicali	9
2.3	Classificatore basato su SVM lineari e word embedding	9
3	Fine-tuning di un modello di linguaggio neurale (Neural Language Model)	10

1 DATASET

1.1 SUBSET DEL DATASET

Il dataset come da richiesta è stato preso dal primo sottotask (sub-task A) di *EVALITA 2026*, ha il formato di una tabella csv composta da una colonna che contiene una serie di testi e la seconda contenente il relativo label di classificazione binaria che identifica se il testo contenuto sia stato generato da un umano (0) o da una macchina (1). Sul sito del dataset erano presenti tre file scaricabili. Il primo file di training composto da una tabella csv con più di 33 mila entry, da questa sono stati richiesti due subset uno di training da 2000 righe e uno di validation da 1000, entrambi bilanciati, quindi con egual numero di label 0 e 1. Gli altri due file erano di test, uno contenente solo la colonna dei testi da classificare e l'altro contenente le soluzioni corrette del test set. Dal test set con label è stato richiesto di estrarre 1000 item anch'essi bilanciati tra label 0 e 1. L'estrazione bilanciata è avvenuta con l'ausilio di due contatori uno per ciascuna label di classificazione, come si può vedere dalla sezione di codice sotto riportata. Per ottenere il bilanciamento tra classi la logica di selezione delle entry prevede che i contatori abbiano lo stesso valore al termine dell'iterazione. A partire dal primo elemento della tabella csv, se il valore dei contatori è uguale viene selezionata l'entry corrente per far parte del subset e incrementato il contatore corrispondente alla label nella seconda colonna, altrimenti dall'elemento corrente si scorrono le successive entry fino a trovare la prima che abbia label corrispondente al counter di valore minore.

```
1 #[...]
2     for riga in reader:
3         #riga contiene il ['test'] e la ['label']
4         if c0+c1 < row:
5             if c0==c1:
6                 match riga["label"]:
7                     case "1":
8                         c1+=1
9                     case "0":
10                        c0+=1
11                subset.append(riga)
12                continue
13            elif c0>c1 and riga["label"] == "1":
14                c1+=1
15                subset.append(riga)
16                continue
17            elif c0<c1 and riga["label"] == "0":
```

```

18             c0+=1
19             subset.append(riga)
20             continue
21         else:
22             break
23     #[...]

```

Per l'estrazione del validation set la logica è la stessa, ma per non avere duplicati il file è stato letto al contrario, cioè a partire dall'ultima entry, infatti dato che ci sono più di 33mila campioni è improbabile che con la selezione del subset di training si arrivi in fondo alla tabella. Nella sezione di codice sotto mostro come è stata invertita la tabella.

```

1     #[...]
2     for riga in reversed(righe):
3     #[...]

```

Tuttavia trattandosi di un evento altamente improbabile ma non impossibile (non ho letto il dataset), volevo essere sicura non ci fossero elementi in comune, perciò ho performato ulteriore verifica che il training set e il validation set non avessero righe in comune. Il check è stato eseguito direttamente da terminale con un piccolo script, riportato di seguito, il quale ha confermato l'assenza di sovrapposizioni.

```

1     import pandas as pd
2
3     df1 = pd.read_csv('train_subset.csv')
4     df2 = pd.read_csv('validation_subset.csv')
5
6     # L'inner merge tiene solo ciò che è presente in ENTRAMBI i dataframe
7     if pd.merge(df1,df2, how='inner'):
8         print("ci sono elementi in comune")
9     else:
10        print("non ci sono elemnti in comune")

```

1.2 FEATURES

Ngrammi La prima rappresentazione del testo richiesta dalla traccia del progetto è la rappresentazione a ngrammi di vario tipo: caratteri, parole, lemmi e PartOfSpeech. Le mie scelte progettuali sono state: *json* per salvare gli ngrammi con la propria frequenza relativa nei rispettivi csv, per la normalizzazione del testo ho usato le *regex* e *unicodedata*, quest'ultimo per normalizzare accenti e simboli, e la libreria *spaCy* per le operazioni di tokenizzazione, lemmatizzazione e PartOfSpeech tagging. La pipeline del processo si può riassumere come:

1. Parte da una riga CSV: text,label
2. Normalizza il testo

3. Analizza il testo con spaCy
4. Divide il testo in frasi
5. Divide le frasi in token
6. Elimina spazi, punteggiatura e stop word
7. Per ogni token salva word, lemma e POS
8. Costruisce un oggetto Document
9. Estrae dal Document parole, lemmi e POS
10. Ricostruisce il testo processato con le sole parole rimaste
11. Estrae char n-gram da quel testo
12. Estrae word n-gram dalla lista di parole
13. Estrae lemma n-gram dalla lista di lemmi
14. Estrae POS n-gram dalla lista di POS
15. Conta gli n-grammi con Counter
16. Converte i conteggi in frequenze relative
17. Trasforma tutto in JSON e salva una riga nel CSV finale

Le strutture dati fondamentali sono la classe **Token** che rappresenta una singola parola/token del testo, di ciascun token vengono salvati la parola stessa, il lemma corrispondente e l'etichetta POS corrispondente alla categoria grammaticale. La seconda classe fondamentale usata è la classe **Sentence** che contiene il testo della frase e la lista di token, più tutta una serie di metodi utili per la gestione, ad esempio poter aggiungere un token, richiedere la lista dei lemmi o quella delle etichette POS. La terza classe è la classe **Document** che rappresenta un documento intero salvandone path, id, di che split fa parte (train, test, validation), la label di classificazione, la lista delle frasi e la lista delle features che inizialmente è un dizionario vuoto. In questo specifico caso abbiamo un'istanza di Document per ogni testo nel csv, e come per le altre classi viene accompagnato da tutta una serie di metodi utili per l'uso della classe. La pipeline parte dalla funzione `create_ngram_csv()` che prima di tutto carica i documenti dalla tabella csv e li trasforma in oggetti Document, poi costruisce il dataframe in cui verranno salvati tutti i formati e tipi di ngrammi, e, infine, salva il nuovo csv. Il caricamento dei documenti avviene tramite la funzione `load_documents_from_csv()` che, usando la libreria *pandas*, carica tutto il file in un dataframe, controllando che ci siano entrambe le colonne (text e label). Poi carica il modello spaCy che di default è `it_core_news_sm`, cioè il modello italiano small ottimizzato per la CPU, addestrato sui testi di Wikipedia e Universal Dependencies. La tabella csv viene letta riga per riga e normalizzata e poi viene chiamata la funzione che costruisce, per ogni riga, gli oggetti Document. Il testo viene anche normalizzato attraverso la funzione `normalize_text()` che prende il testo grezzo e con *unicodedata* normalizza accenti e simboli se presenti, e con `text = re.sub(r\s+", " ", text)` sostituisce spazi multipli, tab e newline con uno spazio singolo. Ho ritenuto opportuno non normalizzare le maiuscole, quindi il testo non viene mai trasformato in lowercase. Il testo così normalizzato passa alla funzione `build_document_from_text()` che lo trasforma in un oggetto istanza di Document e con `nlp(text)` fa la divisione in frasi,

tokenizzazione, lemmatizzazione, etichettatura POS e riconoscimento delle *stop-words*. Per ogni frase del testo crea la rispettiva istanza di Sentence e poi scorre i token della frase, spazi e punteggiatura non vengono considerati come token perciò vengono saltati, così come le stop-words, per ogni token rimasto da questo processo viene creata un'istanza della classe Token, che oltre la parola contiene anche il lemma e l'etichetta POS. Da qui il processo procede al ritroso, l'istanza di ogni token viene aggiunta all'istanza di Sentence tramite il metodo `add_token()`, quando la frase viene completata questa viene riaggiunta all'istanza di Document corrispondente tramite il metodo di classe `add_sentence()`. Dopo aver costruito tutti i Documents lo script chiama `build_ngram_dataframe()` questa funzione prende la lista dei documenti e li trasforma in una riga del dataframe finale. La funzione che si occupa della trasformazione è:

```
1     def build_row_from_document(document):
2         words = document.get_words()
3         lemmas = document.get_lemmas()
4         pos_tags = document.get_pos_tags()
5
6         processed_text = " ".join(words)
7
8         row = {}
9
10        row["text"] = processed_text
11
12        for n in range(2, 5):
13            column_name = f"char_{n}grams"
14            row[column_name] = json.dumps(
15                extract_char_ngrams_from_text(processed_text, n, relative=True),
16                ensure_ascii=False
17            )
18
19        for n in range(1, 5):
20            column_name = f"word_{n}grams"
21            row[column_name] = json.dumps(extract_sequence_ngrams(words, n, relative=True),
22                ensure_ascii=False
23            )
24
25        for n in range(1, 5):
26            column_name = f"lemma_{n}grams"
27            row[column_name] = json.dumps(extract_sequence_ngrams(lemmas, n, relative=True),
28                ensure_ascii=False
29            )
30
31        for n in range(1, 5):
32            column_name = f"pos_{n}grams"
33            row[column_name] = json.dumps(
34                extract_sequence_ngrams(pos_tags, n, relative=True),
35                ensure_ascii=False
36            )
37
```

```
38         row["label"] = document.label
39
40     return row
```

Lavora estrendo prima le tre liste (parole/lemmi/POS) e poi costruisce il testo processato da inserire nella colonna del csv finale. Gli ngrammi di caratteri vengono estratti da `extract_char_ngrams_from_text()`, gli ngrammi di parole si ricavano con `extract_sequence_ngrams()`. La seconda funzione in particolare prende in input una lista, quindi per estrarre i lemmi riuso la stessa funzione, ma passando la lista dei lemmi ottenuta all'inizio della funzione (`lemmas = document.get_lemmas()`) e lo stesso per gli ngrammi di etichette POS. A ciascuno degli ngrammi sia di caratteri che di parole/lemmi/POS viene associata la loro frequenza, in questo caso relativa, tramite il settaggio della flag `relative=True`. Gli ngrammi vengono contati con l'impiego di `Counter()` che poi tramite la funzione `counter_to_json_list()` viene trasformato in una lista json associando ogni contatore al rispettivo ngramma per ciascun documento e calcolandone la frequenza relativa.

2 SVILUPPO E DISCUSSIONE

2.1 CLASSIFICATORE BASATO SU SVM LINEARI E N-GRAMMI

Il programma che si occupa di eseguire la classificazione tramite Linear-SVM basato su ngrammi è `svm_ngrams.py` il quale prende il file csv di training già preparato con la rappresentazione ad ngrammi come illustrato nella Sezione 1.2. Lo script costruisce diversi modelli e, come da richiesta progettuale, li valuta tramite cross-validation stratificata a 5 fold. Vengono aggiunte anche delle metriche che ritengo utili per valutare un modello di classificazione binaria, quali:

- **Accuracy**: è la metrica che misura la percentuale di previsioni corrette effettuate da un modello rispetto al totale. È calcolata come il rapporto tra le previsioni corrette e il totale delle previsioni.
- **F1-Score**: è la media armonica tra Precisione (capacità di non commettere falsi positivi) e Recall/Richiamo (capacità di individuare tutti i casi positivi). Il punteggio varia da 0 (pessimo) a 1 (perfetto).
- **ROC-AUC**: (Receiver Operating Characteristic - Area Under the Curve) è una metrica di valutazione per modelli di classificazione binaria. Misura la capacità del modello di distinguere tra le classi, rappresentando graficamente il rapporto tra veri positivi e falsi positivi al variare della soglia di classificazione. Il suo punteggio è compreso tra 0 e 1.

I risultati vengono salvati su un file Excel. Il progetto richiedeva proprio di sperimentare diverse configurazioni di features e di selezionare poi il modello con le performance migliori. La una delle classi usate, sulla base di come ho deciso di rappresentare le feature di ngrammi¹, è `NgramJSONVectorizer`, Il problema della SVM è il non poter lavorare direttamente su stringhe JSON, ma ha bisogno di una matrice numerica in input. Quindi il vettorizzatore json fa questa trasformazione da stringa a dizionario python e poi `DictVectorizer` prende questo dizionario e lo trasforma in una matrice numerica sparsa in modo da poterla dare come input alla SVM. La funzione `get_ngram_representations()` definisce tutti i gruppi di feature che la SVM deve testare. Oltre alle features singole ho definito anche cinque configurazioni extra che sono:

- `char_2_3_4grams` : tutte le rappresentazioni dal bigramma al 4-gramma di caratteri.
- `word_1_2_3_4grams` : tutte le le rappresentazioni dall'unigramma al 4-gramma di parole.
- `lemma_1_2_3_4grams` : tutte le le rappresentazioni dall'unigramma al 4-gramma di lemmi.

¹Perché nel CSV ogni colonna di n-grammi contiene celle in formato JSON.

- pos_1_2_3_4grams : tutte le le rappresentazioni dall'unigramma al 4-gramma di etichette POS.
- all_ngrams : tutte le rappresentazioni prese insieme.

In totale sono 20 rappresentazioni e per ciascuna vengono calcolati 6 valori di c . Il parametro C controlla la forza della regolarizzazione in modo inversamente proporzionale, la classe `GridSearchCV` di *scikit-learn* permette di testare automaticamente combinazioni diverse di valori per trovare quello ottimale. Più è basso il valore di C più la regolarizzazione è forte e il modello tenderà a semplificare penalizzando i pesi elevati, per valori alti di C , invece, il modello impone una regolarizzazione debole o nulla, in questo modo il modello si adatta maggiormente ai dati di training cercando di minimizzare l'errore, con un alto rischio di overfitting. Prima di passare in input la matrice uso `StandardScaler(with_mean=False)` in modo da rendere le feature confrontabili in scala logaritmica. L'uso del parametro `with_mean=False` mantiene la matrice sparsa: è necessario perchè la matrice degli ngrammi è sparsa, cioè contiene tantissimi zeri: per esempio, un documento che usa solo una piccola parte di tutti gli ngrammi possibili, se si sottraesse la media, la matrice perderebbe la sua sparsità e diventerebbe molto più pesante in memoria. La SVM lineare lavora con la funzione

$$f(x) = wx + b$$

, con w : pesi delle features, b : bias e x : vettore del testo, se

$$f(x) > 0$$

allora il documento viene assegnato alla classe **1**, **0** altrimenti. La SVM cerca la separazione lineare, quindi la retta, con margine massimo, cioè quella che meglio separa e tiene più distanti possibile le due classi dal confine decisionale. Il confine decisionale è rappresentato dall'equazione

$$wx + b = 0$$

, e sono proprio i documenti che si trovano più vicini a questa zona di confine che determinano più incisivamente la posizione dell'iperpiano di separazione, infatti vengono chiamati **support vectors**. Per la cross-validation a 5 fold nello script uso:

```

1
2     StratifiedKFold(
3         n_splits=5,
4         shuffle=True,
5         random_state=42 #il seed che uso sempre come easter-egg di Guida Galattica per Autostoppisti
6     )

```

Ho scelto di usare `StratifiedKFold` anzichè il `K-Fold` normale perchè quando fa la divisione del dataset cerca di mantenere in ogni fold la stessa proporzione tra le classi presente nel dataset originale. Il dataset è stato creato specificatamente bilanciato 50/50, per cui anche nella divisione a 5 parti si mantiene il bilanciamento tra le classi. Ogni modello viene valutato 5 volte sulle 5 fold e per ciascuno di essi il codice salva l'accuracy, f1-score e roc-auc e poi calcola l'accuracy media e la deviazione standard tra i vari risultati e così per tutte e 3 le metriche. Tramite l'impiego di `GridSearchCV` vengono provati automaticamente sei valori di

C^2 , per ciascuno di questi viene eseguita una 5-fold cross-validation, e poi vengono riportati tutti in tabella. Siccome non volevo avere salvati 121 modelli diversi prima di sapere qual è il migliore ho impostato `refit = False`, questo significa che alla fine non viene riaddestrato automaticamente un modello finale su tutto il training set, il programma produce solo la tabella comparativa dei risultati, quindi ogni riga è identificativa della configurazione usata e delle sue performance.

Scelta del modello migliore. La selezione del modello migliore è stata effettuata considerando principalmente la F1 macro media sui 5 fold, affiancata da accuracy media, ROC-AUC media e deviazione standard delle metriche. La F1 macro è stata privilegiata perché consente di valutare il comportamento del classificatore su entrambe le classi del task, evitando di selezionare modelli apparentemente accurati ma sbilanciati verso una sola classe. Il modello migliore risulta essere il modello con rappresentazione a unigrammi di parola e parametro $C = 0.001$. Il modello ottiene accuracy media pari a 0.9465, F1 media pari a 0.9465 e ROC-AUC media pari a 0.9869. Poiché le configurazioni con la stessa rappresentazione e valori più alti di C ottengono risultati identici, è stato selezionato il valore di C più basso, in quanto maggiormente regolarizzato. Tra gli altri modelli efficaci si osservano buone prestazioni per `char_2_3_4grams`, `lemma_1grams` e `char_3grams`. Il modello peggiore è invece quello basato su quadrigrammi di parole, che ottiene accuracy media pari a 0.5000 e F1 macro media pari a 0.3333, indicando un comportamento vicino al caso e fortemente sbilanciato nella classificazione. La scelta di usare F1 score come metrica più importante nella selezione del modello è data dal tipo di task. Abbiamo un compito di decisione binaria, la F1 macro penalizza i modelli che funzionano bene solo su una classe e non sull'altra. dato che il dataset è bilanciato, anche l'accuracy è informativa, però la F1 macro rimane più prudente, perché controlla meglio l'equilibrio tra le due classi. La metrica valutata in seconda è la ROC-AUC che misura la capacità del modello di separare le due classi, ma non garantisce nulla sulla bontà di risposta del modello. I modelli che usano monogrammi di parole sono quelli che in questo caso rendono meglio.

Ruolo	Modello	Rappresentazione	C	Accuracy mean	Accuracy std	F1 Score mean	F1 Score std	ROC-AUC mean	ROC-AUC std
Migliore	model_25	word_1grams	0.001	0.9465	0.0101	0.9465	0.0101	0.9869	0.0023
Buono	model_19	char_2_3_4grams	0.001	0.9440	0.0137	0.9440	0.0137	0.9811	0.0055
Buono	model_55	lemma_1grams	0.001	0.9420	0.0113	0.9420	0.0113	0.9862	0.0040
Buono	model_7	char_3grams	0.001	0.9400	0.0147	0.9400	0.0148	0.9770	0.0062
Peggior	model_43	word_4grams	0.001	0.5000	0.0000	0.3333	0.0000	0.5762	0.0076

Table 2.1. Confronto tra il modello migliore, tre modelli intermedi validi e il modello peggiore. Sono riportate le tre metriche principali calcolate tramite 5-fold cross-validation, insieme alla rispettiva deviazione standard.

²0.001, 0.01, 0.1, 1, 10, 100

- 2.2 CLASSIFICATORE BASATO SU SUPPORT VECTOR MACHINE (SVM) LINEARI E INFORMAZIONI LINGUISTICHE NON LESSICALI
- 2.3 CLASSIFICATORE BASATO SU SVM LINEARI E WORD EMBEDDING

3 FINE-TUNING DI UN MODELLO DI LINGUAGGIO NEURALE (NEURAL LANGUAGE MODEL)